SmartPlantBase · Matera-Gotchi

Matera-Gotchi: Sistema de Gamificación y Computación Afectiva para SmartPlantBase

Asignatura: Arquitectura de Software · **Semestre:** 2026-2 · **Estudiantes:** Esteban Castro, Natalia Moreno, Sebastian Gonzalez · **Núcleo:** Arquitectura Hexagonal

Matera-Gotchi en Vivo · ¡Estoy feliz!
"Gracias por cuidarme, mis parámetros están en armonía."

Nivel 5 (Guardián) | 850 XP | Racha: 12 días | Felicidad: 85%

1. Resumen y Concepto: De Telemetría Fría a Mascota Digital

En la Fase 2 de **SmartPlantBase** implementamos una arquitectura hexagonal completa que procesa telemetría de sensores físicos (humedad del suelo en %, temperatura ambiental en °C y luminosidad en lux) y la evalúa contra rangos biológicos según la especie vegetal. Los datos se transmiten en tiempo real mediante WebSockets a un panel web.

Aunque el backend resuelve con solidez el diagnóstico biológico, la experiencia de usuario (*UX*) revela una barrera crítica: las métricas numéricas y las alertas técnicas terminan siendo ignoradas por el usuario hasta que la planta muere. Para resolverlo, esta propuesta integra una **mecánica de interacción tipo Tamagotchi**, humanizando los datos de telemetría y transformando la matera inteligente en un organismo afectivo y lúdico:

EL PROBLEMA
Las lecturas numéricas y advertencias frías son ignoradas o postergadas por el usuario hasta que el daño vegetal es irreversible.

LA SOLUCIÓN
Antropomorfismo botánico: la planta cobra vida, habla en primera persona con diálogos adaptados y expresa sus emociones.

MECÁNICA CENTRAL
Los hábitos diarios de cuidado se convierten en progresión tangible: puntos de experiencia (XP), niveles crecientes y rachas de días.

Principios Fundamentales del Sistema

1. **Computación Afectiva y Estado Anímico Visual:** En lugar de desplegar solo cifras estáticas, la planta reacciona con expresiones gráficas (emojis y caras ASCII) y mensajes en primera persona según su estado físico (radiante, contenta, sedienta, ahogada, sofocada, congelada, agradecida o marchita).
2. **Progresión por Hábitos Saludables (XP y Rachas):** Mantener la planta en rangos óptimos genera puntos de experiencia (*XP*) y consolida días de racha consecutivos. El descuido continuado penaliza la felicidad y reinicia la racha, incentivando la constancia sin frustrar el avance histórico acumulado.
3. **Reacción Reactiva Inmediata a Acciones Físicas:** Cuando el usuario riega la planta o la expone a la luz requerida, el sensor capta el cambio en tiempo real y la planta responde inmediatamente con el estado `AGRADECIDA` y animaciones festivas en el frontend.
4. **Aislamiento Arquitectónico Estricto (RA3, RA4):** La gamificación se concibe como un **subdominio desacoplado**. La clase de dominio biológico `ValidadorDeEstado` permanece inalterada; la lógica lúdica es coordinada por un servicio especializado (`MotorGamificacion`) mediante puertos y adaptadores dedicados.

PIPELINE IOT Y FLUJO DE DATOS EN TIEMPO REAL (EXTREMO A EXTREMO EN <5 MS)

1. **Ingesta IoT** — ESP32 envía telemetría cruda vía HTTP POST al backend.
2. **Dominio Biológico** — `ValidadorDeEstado` genera diagnóstico puro según especie.
3. **Motor Lúdico** — `MotorGamificacion` calcula emoción, delta XP y diálogo.
4. **Persistencia** — `GamificacionPort` persiste perfil en SQLite relacional.
5. **Streaming WS** — WebSocket emite bloque `tamagotchi` hacia `user_{id}`.

2. Mecánica del Tamagotchi: Estados Anímicos y Progresión

2.1 Matriz de Estados Anímicos y Emociones

El estado anímico se determina de manera **determinista** evaluando el diagnóstico biológico del sensor, los límites fisiológicos de la especie vegetal y la tasa de cambio de humedad (detección de riego):

Estado Anímico	Emoji	Expresión ASCII	Disparador Biológico / Físico	Mensaje al Usuario en Primera Persona	Impacto XP / Felicidad
RADIANTE	🌸✨	(˘ ³˘)♥	Parámetros óptimos y racha > 3 días consecutivos.	*"¡Estoy en mi mejor momento! El sol y el agua están perfectos."*	+25 XP / 100%
CONTENTA	🌿😊	(✿◠‿◠)	Todos los parámetros dentro del rango biológico saludable.	*"Me siento muy bien. Gracias por cuidar de mí con constancia."*	+10 XP / 85%
SEDIENTA	💧🥺	(•̀⌒•́)	Humedad del sustrato en nivel `BAJO`; solicita riego urgente.	*"Tengo sed; mi tierra está bastante seca. ¿Me das un poco de agua?"*	-5 XP / 40%
AHOGADA	🌊🤢	(×_×;)	Humedad saturada `ALTO`; riesgo inminente de pudrición radicular.	*"¡Mucha agua! Mis raíces se están asfixiando, déjame secar un poco."*	-10 XP / 30%
SOFOCADA	☀️🥵	(;´д`)	Estrés ambiental por temperatura alta o iluminación excesiva.	*"Hace demasiado calor aquí. ¿Podrías ubicarme en un lugar fresco?"*	-10 XP / 35%
CONGELADA	❄️🥶	(>_<)	Estrés térmico por temperatura ambiental baja.	*"¡Qué frío! Mis hojas se están entumeciendo, protégeme del viento."*	-10 XP / 35%
AGRADECIDA	💚🥰	(つ✧ω✧)つ	Salto súbito de humedad tras riego en tiempo real (> 15% en < 5 min).	*"¡Agua fresca! Justo lo que necesitaba para revivir. ¡Muchas gracias!"*	+50 XP / +30%
MARCHITA	🥀💀	(x_x)	Diagnóstico crítico multisensor con hojas visiblemente decaídas.	*"Mis hojas están decaídas y débiles... Necesito atención urgente."*	-30 XP / 10%

2.2 Progresión por Niveles y Rachas de Días

El sistema incentiva el hábito continuo mediante 5 niveles botánicos bien diferenciados:

- **Nivel 1: Semilla** (0 - 150 XP) — Avatar brotando, respuestas básicas, tutorial inicial.
- **Nivel 2: Plántula** (151 - 450 XP) — Primeras hojas tiernas, desbloqueo de emociones contextuales.
- **Nivel 3: Planta Joven** (451 - 1000 XP) — Follaje robusto, desbloquea consejos de cultivo avanzados.

- **Nivel 4: Planta Florecida** (1001 - 2000 XP) — Flores visibles y resistencia a un descuido leve sin perder racha.
- **Nivel 5: Guardián Botánico** (2000+ XP) — Planta plenamente desarrollada, halo dorado e insignia de cuidador maestro.

Regla de Racha: La racha se incrementa en +1 cada 24 horas continuas en estado `SALUDABLE`. En caso de diagnóstico `CRITICO`, la racha cae a 0 para reflejar el descuido, pero la XP histórica nunca decrece por debajo de cero, protegiendo la motivación del alumno.

2.3 Personalidad y Diálogos Adaptados por Especie

- **Sansevieria:** Firme, sobria y tolerante a la sequía. Advierte con severidad el exceso de agua: *"Recuerda que soy de clima seco; no acumules agua en mi plato"*.
- **Potos:** Expresiva, sensible a la penumbra. Pide reubicación luminosa si la luxometría permanece baja durante horas.
- **Suculenta:** Almacén hídrico foliar. Entra en alarma temprana si se detecta saturación prolongada.
- **Helecho:** Amante de la humedad constante. Sus mensajes de auxilio hídrico son más inmediatos y expresivos.

---

3. Especificación Formal de Requisitos (RF & RNF)

Los requisitos formales garantizan aislamiento arquitectónico, integridad operativa y confiabilidad:

REQUISITOS FUNCIONALES (RF)

- **RF-01 (Cálculo Determinista de Emociones):** Derivar el estado anímico a partir de la telemetría física y el `Diagnostico` botánico.
- **RF-02 (Acumulación de XP y Niveles):** Conceder +10 XP por ciclo en rango saludable y +25 XP en radiante. Subir de nivel según la tabla de 5 etapas.
- **RF-03 (Detección Reactiva de Riego):** Identificar incrementos súbitos de humedad (> 15% delta) para detonar estado `AGRADECIDA` (+50 XP).
- **RF-04 (Persistencia de Perfil):** Guardar `xp_total`, `nivel`, `racha_dias`, `felicidad` y `ultimo_animo`.
- **RF-05 (Notificación WebSocket):** Transmitir payload enriquecido con bloque `tamagotchi` hacia la sala privada `user_{id}`.
- **RF-06 (Consulta REST):** Exponer `GET /api/plantas/{id}/gamificacion` asegurado con token JWT.

REQUISITOS NO FUNCIONALES (RNF)

- **RNF-01 (Subdominio en Python Puro - RA4):** Lógica lúdica sin dependencias web (Flask/SocketIO) ni de BD, 100% testeable con dobles en memoria.
- **RNF-02 (Separación de Responsabilidades - SRP):** `ValidadorDeEstado` permanece inmutable e higiénico. Gamificación encapsulada en `MotorGamificacion`.
- **RNF-03 (Degradación Grácil):** Si la persistencia o cálculo lúdico falla, la telemetría biológica y la respuesta HTTP al ESP32 se completan normalmente.
- **RNF-04 (Desempeño Ultrarrápido):** Ejecución lúdica en memoria en menos de 5 milisegundos por lectura.
- **RNF-05 (Seguridad Multiusuario):** Autorización estricta por pertenencia de planta en JWT.

---

4. Arquitectura Hexagonal y Aislamiento Concéntrico

El subdominio de gamificación se acopla limpiamente mediante el patrón de **Puertos y Adaptadores**, preservando el centro del hexágono intacto:

AISLAMIENTO CONCÉNTRICO: ADAPTADORES DE ENTRADA → DOMINIO PURO → ADAPTADORES DE SALIDA

Adaptadores Entrada
api_dispositivos.py
POST telemetría IoT cruda.
api_front.py
GET consulta lúdica JWT.

→

Dominio Puro (Python Puro RA4)
Entidades Inmutables: `EstadoAnimico`, `ExpresionTamagotchi`, `PerfilGamificacion`.
Servicio de Dominio: `MotorGamificacion` (evaluación de emoción, deltas de XP y racha).
Puerto de Salida: `GamificacionPort` (contrato abstracto agnóstico a SQLite).

→

Adaptadores Salida
gamificacion_bd.py
Persistencia relacional SQLite.
notificador_ws.py
SocketIO emit a `user_{id}`.

4.1 Dominio Puro (`matera/dominio/`)

- **`gamificacion.py`:** Define las entidades inmutables y el servicio de cálculo de emoción y progresión.
- **`puertos/salida.py`:** Incorpora la interfaz abstracta `GamificacionPort` (`obtener_perfil(planta_id)`, `guardar_perfil(perfil)`, `registrar_evento(evento)`).
- **`casos_uso/registrar_lectura.py`:** Orquesta la validación biológica y la gamificación con cláusula `try/except` de degradación grácil.

4.2 Adaptadores de Salida (`matera/adaptadores/salida/`)

- **`gamificacion_bd.py`:** Implementación SQLite del puerto `GamificacionPort`, con mapeo directo de tuplas a dataclasses inmutables.
- **`notificador_ws.py`:** En `notificar_telemetria`, añade la llave `"tamagotchi"` al payload WebSocket emitido.

4.3 Adaptadores de Entrada (`matera/adaptadores/entrada/`)

- **`api_front.py`:** Expone el endpoint `GET /api/plantas/<planta_id>/gamificacion` protegido con JWT.

---

5. Modelo de Datos Relacional y Persistencia SQL

El diseño relacional es **no destructivo** y garantiza **compatibilidad total** con el esquema existente de la Fase 2 (tablas `plantas`, `usuarios`, `lecturas`), aplicando claves foráneas con `ON DELETE CASCADE`:

```
-- data/bd/gamificacion.sql: Esquema relacional no destructivo para gamificación
CREATE TABLE IF NOT EXISTS perfiles_gamificacion (
    planta_id TEXT PRIMARY KEY,
    xp_total INTEGER NOT NULL DEFAULT 0,
    nivel INTEGER NOT NULL DEFAULT 1,
    racha_dias INTEGER NOT NULL DEFAULT 0,
    felicidad INTEGER NOT NULL DEFAULT 100,
    ultimo_animo TEXT NOT NULL DEFAULT 'CONTENTA',
    ultima_actualizacion TEXT NOT NULL,
    FOREIGN KEY (planta_id) REFERENCES plantas(id) ON DELETE CASCADE
);

CREATE TABLE IF NOT EXISTS eventos_gamificacion (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    planta_id TEXT NOT NULL,
    tipo_evento TEXT NOT NULL, -- 'RIEGO', 'NIVEL_UP', 'RACHA_DIAS', 'PENALIZACION'
    xp_ganada INTEGER NOT NULL,
    detalle TEXT NOT NULL,
    fecha_evento TEXT NOT NULL,
    FOREIGN KEY (planta_id) REFERENCES plantas(id) ON DELETE CASCADE
);

CREATE INDEX IF NOT EXISTS idx_eventos_gamif_planta ON eventos_gamificacion(planta_id, fecha_evento DESC);
```

Garantías de Diseño de Base de Datos: 1. **Cero Modificaciones a Tablas Existentes:** Las tablas `plantas`, `lecturas` y `usuarios` no sufren alteración de columnas, blindando los 106 tests existentes.
2. **Integridad Referencial en Cascada:** Si una planta es eliminada, sus perfiles y eventos lúdicos se limpian automáticamente mediante `ON DELETE CASCADE`.
3. **Historial de Auditoría Lúdica:** La tabla `eventos_gamificacion` registra cronológicamente cada bonificación de XP, permitiendo graficar el historial de cuidados.

---

6. Especificación de Contratos de Código e Interfaces

6.1 Dominio Puro: Entidades Inmutables (`matera/dominio/gamificacion.py`)

```
from dataclasses import dataclass
from enum import Enum
from typing import Optional

class EstadoAnimico(str, Enum):
    RADIANTE = "RADIANTE"
    CONTENTA = "CONTENTA"
    SEDIENTA = "SEDIENTA"
    AHOGADA = "AHOGADA"
    SOFOCADA = "SOFOCADA"
    CONGELADA = "CONGELADA"
    AGRADECIDA = "AGRADECIDA"
    MARCHITA = "MARCHITA"

@dataclass(frozen=True)
class ExpresionTamagotchi:
    estado: EstadoAnimico
    emoji: str
    cara_ascii: str
    mensaje: str
    felicidad: int  # Rango determinista 0 a 100

@dataclass(frozen=True)
class PerfilGamificacion:
    planta_id: str
    xp_total: int
    nivel: int
    racha_dias: int
    felicidad: int
    ultimo_animo: EstadoAnimico
    expresion: Optional[ExpresionTamagotchi] = None
```

6.2 Lógica del Motor de Gamificación (`MotorGamificacion`)

```
class MotorGamificacion:
    """Reglas puras de gamificación botánica y cálculo de experiencia."""

    @staticmethod
    def derivar_expresion(especie: str, diagnostico, es_riego: bool = False, racha: int = 0) -> ExpresionTamagotchi:
        if es_riego:
            return ExpresionTamagotchi(EstadoAnimico.AGRADECIDA, "💚🥰", "(つ✧ω✧)つ",
                                       "¡Agua fresca! Justo lo que necesitaba para mis raíces.", 100)
        if diagnostico.estado.value == "CRITICO":
            return ExpresionTamagotchi(EstadoAnimico.MARCHITA, "🥀💀", "(x_x)",
                                       "Mis hojas están decaídas y débiles... Necesito atención urgente.", 10)
        for p in diagnostico.parametros:
            if p.nombre == "humedad" and p.estado.value == "BAJO":
                return ExpresionTamagotchi(EstadoAnimico.SEDIENTA, "💧🥺", "( •́ ^ •̀ )",
                                           "Tengo sed; mi sustrato está seco. ¿Me das un poco de agua?", 40)
            if p.nombre == "humedad" and p.estado.value == "ALTO":
                return ExpresionTamagotchi(EstadoAnimico.AHOGADA, "🌊🤢", "(×_×;)",
                                           "¡Mucha agua! Mis raíces se están asfixiando.", 30)
            if p.nombre == "temperatura" and p.estado.value == "ALTO":
                return ExpresionTamagotchi(EstadoAnimico.SOFOCADA, "☀️🥵", "(;´д`)",
                                           "Hace demasiado calor aquí. ¿Me trasladas a la sombra?", 35)
            if p.nombre == "temperatura" and p.estado.value == "BAJO":
                return ExpresionTamagotchi(EstadoAnimico.CONGELADA, "❄️🥶", "( >_< )",
                                           "¡Qué frío! Mis hojas se están entumeciendo.", 35)
        if racha >= 3:
            return ExpresionTamagotchi(EstadoAnimico.RADIANTE, "🌸✨", "( ˘ ³˘)♥",
                                       "¡Estoy en mi mejor momento! El sol y el agua están perfectos.", 100)
        return ExpresionTamagotchi(EstadoAnimico.CONTENTA, "🌿😊", "(✿◠‿◠)",
                                   "Me siento muy bien. Gracias por cuidar de mí con constancia.", 85)

    @staticmethod
    def actualizar_perfil(perfil: PerfilGamificacion, exp: ExpresionTamagotchi, es_riego: bool) -> PerfilGamificacion:
        delta_xp = 50 if es_riego else (25 if exp.estado == EstadoAnimico.RADIANTE else (10 if exp.estado ==
EstadoAnimico.CONTENTA else 0))
        nueva_xp = perfil.xp_total + delta_xp
        nuevo_nivel = min(5, 1 + (nueva_xp // 400))
        nueva_felicidad = int((perfil.felicidad * 0.3) + (exp.felicidad * 0.7))
        return PerfilGamificacion(
            planta_id=perfil.planta_id, xp_total=nueva_xp, nivel=nuevo_nivel,
            racha_dias=perfil.racha_dias, felicidad=nueva_felicidad,
            ultimo_animo=exp.estado, expresion=exp
        )
```

6.3 Integración Desacoplada en `RegistrarLectura` y WebSocket Streaming

```
# matera/dominio/casos_uso/registrar_lectura.py
class RegistrarLectura(RegistrarLecturaPort):
    def registrar(self, dispositivo_id: str, valores: Mapping[str, float], marca_tiempo: str = "") -> Diagnostico:
        # 1. Validación biológica y persistencia médica pura (invariable)
        diagnostico = ValidadorDeEstado.evaluar(lectura, especie_entidad)
        self._historial.guardar_lectura(lectura, diagnostico)

        # 2. Subdominio lúdico con cláusula de degradación grácil
        datos_tamagotchi = None
        if self._gamificacion is not None:
            try:
                perfil = self._gamificacion.obtener_perfil(planta.id)
                es_riego = self._detectar_riego(valores, planta.id)
                exp = MotorGamificacion.derivar_expresion(planta.especie, diagnostico, es_riego, perfil.racha_dias)
                perfil = MotorGamificacion.actualizar_perfil(perfil, exp, es_riego)
                self._gamificacion.guardar_perfil(perfil)
                datos_tamagotchi = perfil
            except Exception as err:
                logger.warning("Degradación grácil activada (gamificación): %s", err)

        # 3. Notificación WebSocket con payload enriquecido en < 5ms
        if self._notificador is not None:
            self._notificador.notificar_telemetria(planta, lectura, diagnostico, datos_tamagotchi)
        return diagnostico
```

6.4 Payload JSON Enriquecido en WebSocket (`telemetria_actualizada`)

```
{
  "planta_id": "planta-01",
  "planta_nombre": "Mi Monstera Deliciosa",
  "dispositivo_id": "esp32-matera-01",
  "especie": "monstera",
  "valores": { "humedad": 58.2, "temperatura": 22.4, "luz": 620.0 },
  "estado": "SALUDABLE",
  "tamagotchi": {
    "animo": "CONTENTA",
    "emoji": "🌿😊",
    "cara_ascii": "(✿◠‿◠)",
    "mensaje": "Me siento muy bien. Gracias por cuidar de mí con constancia.",
    "felicidad": 85,
    "xp_total": 850,
    "nivel": 5,
    "racha_dias": 12
  }
}
```

7. Widget Frontend y Experiencia Interactiva (Vanilla JS)

Siguiendo el principio de **cero dependencias externas en frontend (HTML5/CSS3/Vanilla JS)**, el componente visual se integra armónicamente en el panel de control:

```
<!-- Widget Tamagotchi integrado en live-panel (frontend/index.html) -->
<div class="card tamagotchi-card mood-CONTENTA" id="tamagotchi-widget">
    <div class="gotchi-header">
        <span class="gotchi-badge" id="gotchi-mood-tag">CONTENTA</span>
        <span class="gotchi-level-pill" id="gotchi-level-tag">Nivel 5 · Guardián</span>
    </div>
    <div class="gotchi-body">
        <div class="gotchi-avatar-wrap">
            <div class="gotchi-icon" id="gotchi-avatar">🌿</div>
            <div class="gotchi-ascii" id="gotchi-face">(✿◠‿◠)</div>
        </div>
        <div class="gotchi-speech">
            <p id="gotchi-message">"Me siento muy bien. Gracias por cuidar de mí con constancia."</p>
            <div class="gotchi-meters">
                <div class="meter-row">
                    <span>Felicidad: <strong id="gotchi-happy-val">85%</strong></span>
                    <div class="meter-bar"><div class="fill fill-happy" id="gotchi-happy-bar" style="width: 85%;"></div></div>
                </div>
                <div class="meter-row">
                    <span>Experiencia: <strong id="gotchi-xp-val">850 XP</strong></span>
                    <div class="meter-bar"><div class="fill fill-xp" id="gotchi-xp-bar" style="width: 70%;"></div></div>
                </div>
            </div>
        </div>
    </div>
</div>
```

```
/* Animaciones Reactivas en CSS Puro (frontend/css/style.css) */
.gotchi-icon { font-size: 2.8rem; animation: breathing 3s ease-in-out infinite; }
@keyframes breathing { 0%, 100% { transform: translateY(0) scale(1); } 50% { transform: translateY(-3px) scale(1.05); } }
.mood-SEDIENTA .gotchi-icon, .mood-MARCHITA .gotchi-icon { animation: droop 2s ease-in-out infinite; filter: grayscale(40%); }
@keyframes droop { 0%, 100% { transform: rotate(0deg); } 50% { transform: rotate(-6deg) translateY(2px); } }
.mood-AGRADECIDA .gotchi-icon, .mood-RADIANTE .gotchi-icon { animation: bounce 0.8s ease-in-out infinite alternate; }
@keyframes bounce { 0% { transform: scale(1); } 100% { transform: scale(1.15) translateY(-5px); } }
```

```
// Renderizado Reactivo en frontend/js/ui.js
renderizarTamagotchi(t) {
    if (!t) return;
    document.getElementById("gotchi-avatar").textContent = t.emoji;
    document.getElementById("gotchi-face").textContent = t.cara_ascii;
    document.getElementById("gotchi-mood-tag").textContent = t.animo;
    document.getElementById("gotchi-message").textContent = `"${t.mensaje}"`;
    document.getElementById("gotchi-happy-val").textContent = `${t.felicidad}%`;
    document.getElementById("gotchi-happy-bar").style.width = `${t.felicidad}%`;
    document.getElementById("gotchi-xp-val").textContent = `${t.xp_total} XP`;
    document.getElementById("gotchi-level-tag").textContent = `Nivel ${t.nivel}`;
    const widget = document.getElementById("tamagotchi-widget");
    widget.className = `card tamagotchi-card mood-${t.animo}`;
}
```

8. Plan de Implementación (Roadmap de 5 Hitos)

Roadmap estructurado para entrega continua sin riesgo de regresión sobre los 106 tests actuales:

Hito	Alcance y Tareas Principales	Módulos Modificados / Creados	Criterio de Aceptación y QA
Hito 1	**Dominio Puro de Gamificación** Entidades `EstadoAnimico`, `PerfilGamificacion` y servicio `MotorGamificacion`. Pruebas unitarias sin dependencias externas.	`matera/dominio/gamificacion.py` `tests/test_gamificacion.py`	100% tests pasando en `pytest` en < 0.1s. Cobertura completa de matriz de estados.

Hito 2	**Puertos y Persistencia SQLite**
	Interfaz abstracta `GamificacionPort`, adaptador `gamificacion_bd.py` y script SQL de migración limpia.
		`matera/dominio/puertos/salida.py`
		`matera/adaptadores/salida/gamificacion_bd.py`
		`data/bd/gamificacion.sql`
			CRUD verificado con `ON DELETE CASCADE`. Cero alteraciones a tablas de Fase 2.

Hito 3	**Integración en Caso de Uso y WS** Inyección protegida en `RegistrarLectura` y extensión de `SocketIONotificadorAdapter`.	`matera/dominio/casos_uso/registrar_lectura.py` `matera/adaptadores/salida/notificador_ws.py`	Evento WebSocket enriquecido con latencia de cálculo menor a 5 ms.

Hito 4	**Frontend y Widget Tamagotchi**
	Maquetación HTML en `#live-panel`, animación reactiva CSS (respiración, marchitamiento, salto) y binding en `ui.js`.
		`frontend/index.html`
		`frontend/css/style.css`
		`frontend/js/ui.js`
			El avatar cambia de emoji, expresión ASCII y color dinámicamente sin refrescar la página.

Hito 5	**Simulación de Riego y Verificación E2E** Bandera `--regar` en `simulador_esp32.py` para inducir salto > 15% de humedad y pruebas end-to-end.	`simulador_esp32.py` `tests/test_e2e_gamificacion.py`	Reacción inmediata con estado `AGRADECIDA`, +50 XP y feedback visual instantáneo.

9. Conclusiones y Valor Técnico del Proyecto

1. UX Humanizada

Transforma métricas frías en apego afectivo, resolviendo el problema de abandono del cuidado botánico mediante micro-hábitos y retroalimentación inmediata.

2. Clean Architecture

Valida la extensibilidad de los puertos y adaptadores de la Fase 2: el dominio biológico permanece inmaculado y el nuevo subdominio se conecta de forma aislada.

3. Garantía de Calidad

Mantiene el 100% de la suite de 106 pruebas unitarias e integración en verde, garantizando cero regresiones y degradación grácil en fallas.

Balance Técnico del Proyecto: *"Matera-Gotchi demuestra que la arquitectura de software limpia y desacoplada permite extender un sistema IoT con computación afectiva y mecánicas lúdicas de forma ágil, escalable y sin comprometer el rigor biológico ni la estabilidad operativa del sistema."*